

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №5

По дисциплине “Системное программирование”

Специальность “Программная инженерия”

Выполнил:

Я.В. Автушенко

Студент группы ПО-13

Проверил:

А.Д. Кулик

преп.-стажёр кафедры ИИТ

Цель: приобрести практические навыки разработки API и баз данных

Задание 1:

Общее задание

1. Реализовать базу данных из не менее 5 таблиц на заданную тематику.

При реализации продумать типизацию полей и внешние ключи в таблицах;

2. Визуализировать разработанную БД с помощью схемы, на которой отображены все таблицы и связи между ними (пример, схема на рис. 1);

3. На языке Python с использованием SQLAlchemy реализовать подключение к БД;

4. Реализовать основные операции с данными (выборку, добавление, удаление, модификацию);

5. Для каждой реализованной операции с использованием FastAPI реализовать отдельный эндпоинт;

Базу данные можно реализовать в любой СУБД (MySQL, PostgreSQL, SQLite и др.)

1) База данных Деканат

Database.py:

```
from sqlalchemy import create_engine, Column, Integer, String, Date, ForeignKey,
CheckConstraint
```

```
from sqlalchemy.ext.declarative import declarative_base
```

```
from sqlalchemy.orm import sessionmaker, relationship
```

```
from datetime import date
```

```
# Строка подключения к LocalDB
```

```
DATABASE_URL
```

```
"mssql+pyodbc://@(localdb)\\MSSQLLocalDB/DeaneryDB?driver=ODBC+Driver+17+for+SQL+Server&trusted_connection=yes"
```

```
engine = create_engine(DATABASE_URL)
```

```
SessionLocal = sessionmaker(bind=engine)
```

```
Base = declarative_base()
```

```
# Модели
```

```
class Group(Base):
```

```
    __tablename__ = 'Groups'
```

```
    group_id = Column(Integer, primary_key=True)
```

```
    group_name = Column(String(20), nullable=False, unique=True)
```

```
    course = Column(Integer)
```

```
    faculty = Column(String(100), nullable=False)
```

```
    students = relationship("Student", back_populates="group")
```

```
class Student(Base):
```

```
    __tablename__ = 'Students'
```

```
    student_id = Column(Integer, primary_key=True)
```

```
    last_name = Column(String(50), nullable=False)
```

```
    first_name = Column(String(50), nullable=False)
```

```
    middle_name = Column(String(50))
```

```
    birth_date = Column(Date, nullable=False)
```

```
    phone = Column(String(20))
```

```
    email = Column(String(100))
```

```
    group_id = Column(Integer, ForeignKey('Groups.group_id'))
```

```
    group = relationship("Group", back_populates="students")
```

```
    grades = relationship("Grade", back_populates="student")
```

```
class Teacher(Base):
```

```
    __tablename__ = 'Teachers'
```

```
    teacher_id = Column(Integer, primary_key=True)
```

```
    last_name = Column(String(50), nullable=False)
```

```
    first_name = Column(String(50), nullable=False)
```

```
    middle_name = Column(String(50))
```

```
degree = Column(String(50))
```

```
phone = Column(String(20))
```

```
subjects = relationship("Subject", back_populates="teacher")
```

```
class Subject(Base):
```

```
    __tablename__ = 'Subjects'
```

```
    subject_id = Column(Integer, primary_key=True)
```

```
    subject_name = Column(String(100), nullable=False)
```

```
    hours = Column(Integer)
```

```
    teacher_id = Column(Integer, ForeignKey('Teachers.teacher_id'))
```

```
    teacher = relationship("Teacher", back_populates="subjects")
```

```
    grades = relationship("Grade", back_populates="subject")
```

```
class Grade(Base):
```

```
    __tablename__ = 'Grades'
```

```
    grade_id = Column(Integer, primary_key=True)
```

```
    student_id = Column(Integer, ForeignKey('Students.student_id'))
```

```
    subject_id = Column(Integer, ForeignKey('Subjects.subject_id'))
```

```
    grade = Column(Integer)
```

```
    grade_date = Column(Date, default=date.today)
```

```
    student = relationship("Student", back_populates="grades")
```

```
    subject = relationship("Subject", back_populates="grades")
```

main.py:

```
from fastapi import FastAPI, HTTPException, Depends
```

```
from sqlalchemy.orm import Session
```

```
from typing import List, Optional
```

```
from datetime import date
```

```
from pydantic import BaseModel
```

```
from database import SessionLocal, engine, Base, Student, Group, Teacher, Subject, Grade
```

```
app = FastAPI(title="Деканат API")
```

```
# Dependency для получения сессии БД
```

```
def get_db():
```

```
    db = SessionLocal()
```

```
    try:
```

```
        yield db
```

```
    finally:
```

```
        db.close()
```

```
# Pydantic модели для данных
```

```
class StudentCreate(BaseModel):
```

```
    last_name: str
```

```
    first_name: str
```

```
    middle_name: Optional[str] = None
```

```
    birth_date: date
```

```
    phone: Optional[str] = None
```

```
    email: Optional[str] = None
```

```
    group_id: int
```

```
class StudentResponse(StudentCreate):
```

```
    student_id: int
```

```
    class Config:
```

```
        orm_mode = True
```

```
# ===== Эндпойнты =====
```

```
# 1. ПОЛУЧЕНИЕ (SELECT) - список всех студентов
```

```
@app.get("/students", response_model=List[StudentResponse])
```

```
def get_students(db: Session = Depends(get_db)):
```

```
    return db.query(Student).all()
```

2. ДОБАВЛЕНИЕ (INSERT) - нового студента

```
@app.post("/students", response_model=StudentResponse)
```

```
def create_student(student: StudentCreate, db: Session = Depends(get_db)):
```

```
    db_student = Student(**student.dict())
```

```
    db.add(db_student)
```

```
    db.commit()
```

```
    db.refresh(db_student)
```

```
    return db_student
```

3. УДАЛЕНИЕ (DELETE) - студента по ID

```
@app.delete("/students/{student_id}")
```

```
def delete_student(student_id: int, db: Session = Depends(get_db)):
```

```
    student = db.query(Student).filter(Student.student_id == student_id).first()
```

```
    if not student:
```

```
        raise HTTPException(status_code=404, detail="Студент не найден")
```

```
    db.delete(student)
```

```
    db.commit()
```

```
    return {"message": f"Студент {student_id} удален"}
```

4. МОДИФИКАЦИЯ (UPDATE) - обновление данных студента

```
@app.put("/students/{student_id}", response_model=StudentResponse)
```

```
def update_student(student_id: int, student: StudentCreate, db: Session = Depends(get_db)):
```

```
    db_student = db.query(Student).filter(Student.student_id == student_id).first()
```

```
    if not db_student:
```

```
        raise HTTPException(status_code=404, detail="Студент не найден")
```

```
    for key, value in student.dict().items():
```

```
        setattr(db_student, key, value)
```

```
    db.commit()
```

```
    db.refresh(db_student)
```

```
return db_student
```

```
# Дополнительные эндпойнты для демонстрации
```

```
@app.get("/groups/{group_id}/students")
```

```
def get_students_by_group(group_id: int, db: Session = Depends(get_db)):
```

```
    return db.query(Student).filter(Student.group_id == group_id).all()
```

```
@app.get("/teachers")
```

```
def get_teachers(db: Session = Depends(get_db)):
```

```
    return db.query(Teacher).all()
```

```
@app.post("/grades")
```

```
def add_grade(student_id: int, subject_id: int, grade: int, db: Session = Depends(get_db)):
```

```
    new_grade = Grade(student_id=student_id, subject_id=subject_id, grade=grade)
```

```
    db.add(new_grade)
```

```
    db.commit()
```

```
    return {"message": "Оценка добавлена"}
```

Деканат API 0.1.0 OAS 3.1
/openapi.json

default		^
GET	/students Get Students	▼
POST	/students Create Student	▼
DELETE	/students/{student_id} Delete Student	▼
PUT	/students/{student_id} Update Student	▼
GET	/groups/{group_id}/students Get Students By Group	▼
GET	/teachers Get Teachers	▼
POST	/grades Add Grade	▼

Schemas		^
HTTPValidationError	> Expand all object	
StudentCreate	> Expand all object	
StudentResponse	> Expand all object	
ValidationError	> Expand all object	

Вывод: были отработаны базовые знания языка программирования Python при решении практических задач